

Rolling Class Software Setup: Common Steps for Ubuntu and MAC OS

Big Data Algorithms — setup guide

Before you start this guide, you should have completed either “02_software_setup_ubuntu.qmd” (for Ubuntu, including Ubuntu inside WSL on Windows) or “02_software_setup_mac.qmd” (for a Mac). Those guides install R, Python, PostgreSQL, Quarto and Positron. This guide installs the R and Python packages we use in the course and sets up Quarto to run R and Python in the same document. From here on, the steps are the same on Ubuntu and on a Mac.

Run every command in the terminal. A few commands are too long to fit on one line, so they continue on the following lines, either after a \ or inside quotes. Copy all the lines together and the terminal runs them as one command.

We will add packages to this guide during the semester. The last section explains what to do when that happens.

Python environment and packages

Neither Ubuntu nor Homebrew allows `pip` to install packages into the Python they manage (you would get an `externally-managed-environment` error). Instead, create a virtual environment for the course and install packages there:

```
python3 -m venv ~/.venvs/bigdata
source ~/.venvs/bigdata/bin/activate
```

Your prompt now begins with `(bigdata)`, and inside the environment the commands `python` and `pip` refer to the environment’s own copies. Type `deactivate` to leave it. You need to run the `source` line again in every new terminal where you want to use the environment. Positron looks for environments in `~/.venvs`, so it will find this one on its own.

With the environment active, install the Python packages we use in the course, so that everyone works with the same set:

```
pip install numpy pandas polars duckdb pyarrow scikit-learn lightgbm \
  ipykernel "xgboost-cpu; sys_platform == 'linux'" \
  "xgboost; sys_platform == 'darwin'"
```

The last two entries, quotes included, are two versions of the same package, and `pip` picks the one for your system. On Ubuntu it installs `xgboost-cpu`, a build without GPU support; the regular build pulls in several hundred megabytes of NVIDIA libraries that only matter when training on multiple GPUs. On a Mac, where no such build exists or is needed, it installs the regular `xgboost`. Either way the package is used as `xgboost` in your code. Near the start of its output, `pip` prints a line beginning with `Ignoring` about the version it did not install, which is expected.

This downloads a couple of hundred megabytes, so it can take several minutes. Check that the packages load:

```
python -c "import numpy, pandas, polars, duckdb, pyarrow; print('ok')"
```

```
python -c "import sklearn, xgboost, lightgbm; print('ok')"
```

Both commands should print `ok`.

R packages

Install the R packages we use in the course:

```
Rscript -e 'install.packages(c("tidyverse", "data.table", "R.utils",
  "arrow", "duckdb", "RPostgres", "bench", "profvis", "httr2", "digest",
  "future", "furrr", "xgboost", "lightgbm",
  "knitr", "rmarkdown", "reticulate"))'
```

The packages listed need many others, so well over a hundred packages are installed and it takes several minutes. When one package fails, R moves on to the next and only prints a warning at the end, so check that the main ones load:

```
Rscript -e 'suppressMessages({library(tidyverse); library(data.table)
  library(arrow); library(duckdb); library(RPostgres); library(xgboost)
  library(lightgbm); library(reticulate)})
cat("ok\n")'
```

This should print ok. If it says `there is no package called something`, look through the output of the `install` command for a line saying `installation of package ... had non-zero exit status`. That package had to be compiled on your computer and something it needs was missing, almost always a system library. On Ubuntu, the usual cause is having skipped the `sudo apt install` line in the “Prepare R for installing packages” section of the Ubuntu guide; RPostgres, for example, needs `libpq-dev` from that line. Run that line, then run the `install.packages` command above again.

R and Python together in Quarto

A single Quarto document can contain both R and Python code. Quarto runs the code in such a document with R’s `knitr` and `rmarkdown` packages, and the `reticulate` package lets R start Python for the Python parts. All three were installed in the previous section.

Left alone, `reticulate` chooses a Python by itself, and it may not be your `bigdata` environment. Packages you installed there would then appear to be missing when you render a document. Tell `reticulate` which Python to use by setting the variable `RETICULATE_PYTHON` in `~/.Renviron`, a file R reads every time it starts:

```
echo "RETICULATE_PYTHON=$HOME/.venvs/bigdata/bin/python" >> ~/.Renviron
```

Check the result:

```
Rscript -e 'reticulate::py_config()'
```

The first line should read `python:` followed by the path of `.venvs/bigdata/bin/python` inside your home directory. Further down, the `numpy:` line should show a folder inside `.venvs/bigdata` as well, which confirms that R sees the Python packages you installed above.

When the package lists change

When we add a package to one of the lists above, repeat that step:

- For Python, activate the environment with `source ~/.venvs/bigdata/bin/activate` and run the `pip install` command again. `pip` skips the packages you already have.
- For R, run the `install.packages` command again. R installs all the listed packages again, which also brings them up to date.